

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

*CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting
(BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Constraint Based Problem Solving

Code of Conduct:

*I **THAMIZHARASAN S (192424087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
THAMIZHARASAN S

Annexure A

Short Answer Test

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

ANSWER:

A constraint-based data cleaning pipeline using **NumPy and Pandas** can be designed as follows:

1. Load and inspect the data

```
import pandas as pd
import numpy as np
df = pd.read_csv("patient_records.csv")
print(df.info())
print(df.isnull().sum())
```

2. Handle missing values: Numerical missing values can be replaced with the **median**, while categorical missing values can be replaced with the **mode**.

```
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
```

3. Enforce numerical constraints: Define medically valid ranges and use **filtering and Boolean indexing** to identify invalid values.

```
df.loc[(df['Age'] < 0) | (df['Age'] > 120), 'Age'] = np.nan
df['Age'] = df['Age'].fillna(df['Age'].median())
```

The same method can be applied to blood pressure, heart rate, weight, etc.

4. Standardize categorical data: Remove extra spaces and convert values into a common format.

```
df['Gender'] = df['Gender'].str.strip().str.lower()
df['Gender'] = df['Gender'].replace({'m':'Male', 'male':'Male',
                                     'f':'Female', 'female':'Female'})
```

5. Use aggregation for validation:

```
print(df[['Age', 'HeartRate']].agg(['min', 'max', 'mean', 'count']))
print(df.isnull().sum())
```

Functions such as min(), max(), mean(), and count() verify that numerical values satisfy the constraints and that all records are present.

6. Remove duplicate records if necessary:

```
df = df.drop_duplicates()
```

Thus, indexing (loc) is used to modify specific invalid values, filtering identifies missing/out-of-range values, and aggregation verifies the final dataset. Using imputation instead of deleting complete records preserves maximum data while ensuring no missing values, medically valid numerical values, and standardized categorical fields.

CODE:

```
import pandas as pd

import numpy as np

data = {

    'Patient_ID': [101, 102, 103, 104, 105, 106, 107, 108],

    'Age': [25, 45, np.nan, 130, 60, 35, -5, 50],

    'Weight': [65, 72, 80, 500, np.nan, 68, 70, 75],

    'HeartRate': [72, 80, np.nan, 300, 65, 75, 25, 90],

    'Gender': ['Male', ' male ', 'F', 'female', np.nan, 'M', 'Female', 'f'],

    'BloodGroup': ['A+', 'a+', 'B+', ' O+ ', np.nan, 'AB+', 'b+', 'A+']
```

```
}
```

```
...print("\nCleaning completed successfully!")
```

OUTPUT:

```
else:
    print(f"X {col} contains invalid values")
    valid_ranges = False

print("\nCleaning completed successfully!")
```

```
*** ===== ORIGINAL DATA =====
  Patient_ID  Age  Weight  HeartRate  Gender  BloodGroup
0         101  25.0   65.0       72.0   Male         A+
1         102  45.0   72.0       80.0  male         a+
2         103   NaN   80.0        NaN      F         B+
3         104 130.0  500.0      300.0  female         O+
4         105  60.0    NaN       65.0    NaN         NaN
5         106  35.0   68.0       75.0      M         AB+
6         107  -5.0   70.0       25.0  Female         b+
7         108  50.0   75.0       90.0      f         A+
```

```
===== MISSING VALUES BEFORE CLEANING =====
Patient_ID    0
Age           1
Weight        1
HeartRate     1
Gender        1
BloodGroup    1
dtype: int64

Invalid values in Age: 2

Invalid values in Weight: 1

Invalid values in HeartRate: 2

===== CLEANED DATA =====
  Patient_ID  Age  Weight  HeartRate  Gender  BloodGroup
0         101  25.0   65.0       72.0   Male         A+
1         102  45.0   72.0       80.0   Male         A+
2         103  45.0   80.0       75.0  Female         B+
```

```

2      103  45.0   80.0   75.0 Female      B+
3      104  45.0   71.0   75.0 Female      O+
...
4      105  60.0   71.0   65.0 Female      A+
5      106  35.0   68.0   75.0 Male       AB+
6      107  45.0   70.0   75.0 Female      B+
7      108  50.0   75.0   90.0 Female      A+

```

===== MISSING VALUES AFTER CLEANING =====

```

Patient_ID  0
Age         0
Weight      0
HeartRate   0
Gender      0
BloodGroup  0
dtype: int64

```

===== NUMERICAL DATA VALIDATION =====

```

      Age  Weight  HeartRate
min   25.00   65.0   65.000
max   60.00   80.0   90.000
mean  43.75   71.5   75.875
count    8.00    8.0    8.000

```

===== STANDARDIZED GENDER VALUES =====

```

<StringArray>
['Male', 'Female']
Length: 2, dtype: string

```

===== STANDARDIZED BLOOD GROUP VALUES =====

```

<StringArray>
['A+', 'B+', 'O+', 'AB+']
Length: 4, dtype: string

```

===== FINAL VALIDATION =====

```

✓ No missing values remain
✓ Age values are within valid range
✓ Weight values are within valid range
✓ HeartRate values are within valid range

```

Cleaning completed successfully!

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

ANSWER:

For a massive numerical dataset, **NumPy** can be used efficiently by avoiding unnecessary copies and processing only the required portions of the data.

1. **Efficient array creation:** Use appropriate data types such as float32 instead of float64 to reduce memory usage.

```
import numpy as np
```

```
data = np.random.rand(1000000, 5).astype(np.float32)
print("Memory used:", data.nbytes / (1024**2), "MB")
```

2. **Use slicing instead of copying:** Slicing creates a view of the array in many cases, so it uses less memory.

```
sample = data[:10000]
```

3. **Use in-place operations:** Operations such as += and *= modify the existing array instead of creating another large array.

```
data *= 2
data += 10
```

4. **Random sampling:** Instead of creating another copy of the complete dataset, select only the required indices.

```
indices = np.random.choice(len(data), 1000, replace=False)
sample = data[indices]
```

5. **Optimize indexing:** Boolean indexing can create a new array, so for very large datasets it should be used carefully. Slicing is preferred when possible.

```
part = data[100000:200000]
```

6. **Optimize aggregation:** NumPy aggregation functions such as mean(), sum(), min(), and max() can calculate statistics efficiently.

```
print("Mean:", np.mean(data, axis=0))
print("Maximum:", np.max(data, axis=0))
print("Minimum:", np.min(data, axis=0))
```

For very large datasets, process the data in **chunks** rather than loading everything into memory at once.

7. **Debugging memory problems:** Check the array size using:

```
print(data.nbytes / (1024**2), "MB")
print(data.dtype)
print(data.shape)
```

If memory overflow occurs, reduce the data type size, process the data in smaller chunks, avoid unnecessary copies, and delete unused arrays using:

```
del sample
```

For performance bottlenecks, use Python's time module to measure execution time and identify slow operations.

Conclusion:

The main strategy is efficient array creation + slicing/views + in-place operations + limited random sampling + optimized aggregation. These techniques reduce memory consumption and improve performance. If memory overflow occurs, check nbytes, dtype, and unnecessary array copies; if execution is slow, measure individual operations and optimize the bottleneck.

CODE:

```
import numpy as np

import time

print("===== 1. ARRAY CREATION =====")

data = np.random.rand(1_000_000, 5).astype(np.float32)

print("Shape:", data.shape)

print("Data type:", data.dtype)

print("Memory used:", data.nbytes / (1024**2), "MB")

print("\n===== 2. SLICING =====")

start = time.time()
```

```
sample_slice = data[:100_000]
```

```
...print("✓ Statistical aggregation")
```

```
print("✓ Memory monitoring")
```

```
print("✓ Performance testing")
```

OUTPUT:

```
print("✓ Performance testing")

... ===== 1. ARRAY CREATION =====
Shape: (1000000, 5)
Data type: float32
Memory used: 19.073486328125 MB

===== 2. SLICING =====
Sliced shape: (100000, 5)
Slicing time: 0.00017118453979492188 seconds

===== 3. IN-PLACE OPERATIONS =====
In-place operations completed
Time: 0.004634380340576172 seconds

===== 4. RANDOM SAMPLING =====
Random sample shape: (1000, 5)
Sampling time: 0.03204607963562012 seconds

===== 5. STATISTICAL ANALYSIS =====
Mean: [11.000632 11.000971 10.999693 11.001221 10.999598]
Minimum: [10.000006 10.000007 10.000002 10.          10.000001]
Maximum: [11.999998 11.999998 11.999999 11.999999 11.999999]
Sum: [11000632. 11000971. 10999693. 11001221. 10999598.]
Aggregation time: 0.14888262748718262 seconds

===== 6. MEMORY CHECK =====
Data shape: (1000000, 5)
Data type: float32
Total memory: 19.073486328125 MB

===== 7. INDEXING =====
First row:
[10.78238  10.48714  10.934283 10.058647 11.089801]

First column:
[10.78238  11.555549 10.907575 11.819743 10.168543 11.371264 11.675114
 10.462676 11.350062 10.676486]

===== 8. PERFORMANCE TEST =====
Normal operation time: 0.0069506168365478516 seconds
```



```
===== 8. PERFORMANCE TEST =====
Normal operation time: 0.0069506168365478516 seconds
In-place operation time: 0.0022482872009277344 seconds

===== 9. MEMORY CLEANUP =====
Unused arrays deleted successfully.

===== PROGRAM COMPLETED =====
✓ Efficient array creation
✓ Slicing
✓ In-place operations
✓ Random sampling
✓ Statistical aggregation
✓ Memory monitoring
✓ Performance testing
```

3. Two large Pandas Data Frames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

ANSWER:

Two large Pandas DataFrames can be combined using a **merge/join operation** based on Customer_ID. The merged data should then be validated for duplicate IDs, missing records, and correct indexing.

1. Create and load the DataFrames

```
import pandas as pd
transactions = pd.read_csv("transactions.csv")
demographic = pd.read_csv("demographic.csv")
print(transactions.head())
print(demographic.head())
```

2. Check duplicate Customer IDs

```
print("Duplicate IDs in Transactions:",
      transactions['Customer_ID'].duplicated().sum())

print("Duplicate IDs in Demographic:",
```

```
demographic['Customer_ID'].duplicated().sum())
```

If the demographic table should contain only one record per customer, remove duplicates:

```
demographic = demographic.drop_duplicates(
    subset='Customer_ID',
    keep='first'
)
```

3. Combine the DataFrames

A **left merge** preserves all valid transaction records:

```
merged = pd.merge(
    transactions,
    demographic,
    on='Customer_ID',
    how='left'
)
```

This is useful because every valid transaction is retained even if demographic information is missing.

4. Ensure consistent indexing

```
merged = merged.reset_index(drop=True)
print(merged.index)
```

`reset_index(drop=True)` creates a clean continuous index after merging.

5. Check for duplicate customer IDs

If the final requirement is **one row per customer**, group transactions first:

```
customer_summary = transactions.groupby(
    'Customer_ID',
    as_index=False
)['Amount'].sum()
merged = pd.merge(
```

```
customer_summary,  
demographic,  
on='Customer_ID',  
how='left'  
)  
merged = merged.sort_values('Customer_ID').reset_index(drop=True)
```

This prevents multiple transaction rows from creating duplicate customer records.

6. Validate missing and mismatched entries

```
print("Missing values:")  
print(merged.isnull().sum())
```

Find customers present in transactions but missing from demographics:

```
missing_demo = transactions[  
    ~transactions['Customer_ID'].isin(demographic['Customer_ID'])  
]  
print("Customers missing demographic data:")  
print(missing_demo)
```

7. Validate using grouping

```
duplicates = merged.groupby('Customer_ID').size()  
print("IDs appearing more than once:")  
print(duplicates[duplicates > 1])
```

8. Sort and verify the final DataFrame

```
merged = merged.sort_values(  
    by='Customer_ID'  
)  
.reset_index(drop=True)  
  
print(merged.head())
```

Conclusion

The correct process is:

Load → Check IDs → Remove unwanted duplicates → Merge → Reset index → Filter missing IDs → Group and validate → Sort → Final verification.

Using `merge()` combines the datasets, `drop_duplicates()` prevents duplicate customer IDs, `groupby()` detects repeated IDs, filtering identifies mismatched records, and `sort_values()` with `reset_index()` ensures a consistent and organized final DataFrame. A **left join** is preferred when preserving all valid transaction records is required.

CODE:

```
import pandas as pd

import numpy as np

transactions = pd.DataFrame({

    'Customer_ID': [101, 102, 103, 104, 105, 106, 107],

    'Transaction_Amount': [500, 1200, 750, 300, 900, 450, 1100],

    'Transaction_Type': [

        'Purchase', 'Purchase', 'Refund',

        'Purchase', 'Purchase', 'Purchase', 'Purchase'

    ]

})

... print("X Some transaction records are missing")

missing_count = final_data.isnull().sum().sum()

print("Total missing values:", missing_count)

print("\n===== PROCESS COMPLETED =====")
```

OUTPUT:

```
print("\n===== PROCESS COMPLETED =====")
```

```
*** ===== TRANSACTION DATA =====
```

	Customer_ID	Transaction_Amount	Transaction_Type
0	101	500	Purchase
1	102	1200	Purchase
2	103	750	Refund
3	104	300	Purchase
4	105	900	Purchase
5	106	450	Purchase
6	107	1100	Purchase

```
===== DEMOGRAPHIC DATA =====
```

	Customer_ID	Age	Gender	City
0	101	25	Male	Chennai
1	102	32	Female	Bangalore
2	103	28	Male	Chennai
3	104	45	Female	Mumbai
4	105	35	Male	Delhi
5	108	40	Female	Hyderabad

```
===== DUPLICATE ID CHECK =====
```

Duplicate IDs in Transactions:

Empty DataFrame

Columns: [Customer_ID, Transaction_Amount, Transaction_Type]

Index: []

Duplicate IDs in Demographic:

Empty DataFrame

Columns: [Customer_ID, Age, Gender, City]

Index: []

Duplicates removed from demographic data.

```
===== MISMATCHED IDs =====
```

Customers missing from demographic data:

	Customer_ID	Transaction_Amount	Transaction_Type
5	106	450	Purchase
6	107	1100	Purchase

```
... 5      106      450      Purchase
     6      107      1100     Purchase
```

Customers missing from transaction data:

```
Customer_ID Age Gender City
5      108  40 Female Hyderabad
```

===== MERGING DATA =====

	Customer_ID	Transaction_Amount	Transaction_Type	Age	Gender	City
0	101	500	Purchase	25.0	Male	Chennai
1	102	1200	Purchase	32.0	Female	Bangalore
2	103	750	Refund	28.0	Male	Chennai
3	104	300	Purchase	45.0	Female	Mumbai
4	105	900	Purchase	35.0	Male	Delhi
5	106	450	Purchase	NaN	NaN	NaN
6	107	1100	Purchase	NaN	NaN	NaN

===== DATA AFTER RESETING INDEX =====

	Customer_ID	Transaction_Amount	Transaction_Type	Age	Gender	City
0	101	500	Purchase	25.0	Male	Chennai
1	102	1200	Purchase	32.0	Female	Bangalore
2	103	750	Refund	28.0	Male	Chennai
3	104	300	Purchase	45.0	Female	Mumbai
4	105	900	Purchase	35.0	Male	Delhi
5	106	450	Purchase	NaN	NaN	NaN
6	107	1100	Purchase	NaN	NaN	NaN

===== MISSING VALUES =====

```
Customer_ID      0
Transaction_Amount 0
Transaction_Type  0
Age              2
Gender           2
City             2
dtype: int64
```

===== CUSTOMER ID GROUPING =====

```
- - - - -
```

```

===== CUSTOMER ID GROUPING =====
Customer_ID
... 101      1
     102      1
     103      1
     104      1
     105      1
     106      1
     107      1
dtype: int64

Customer IDs appearing more than once:
No duplicate customer IDs found.

===== TRANSACTION SUMMARY =====
Customer_ID  Transaction_Amount
0           101                500
1           102               1200
2           103                750
3           104                300
4           105                900
5           106                450
6           107               1100

===== FINAL SORTED DATA =====
Customer_ID  Transaction_Amount  Age  Gender  City
0           101                500  25.0   Male  Chennai
1           102               1200  32.0  Female  Bangalore
2           103                750  28.0   Male  Chennai
3           104                300  45.0  Female  Mumbai
4           105                900  35.0   Male  Delhi
5           106                450   NaN   NaN   NaN
6           107               1100   NaN   NaN   NaN

===== FINAL VALIDATION =====
✓ No duplicate Customer IDs
✓ Indexing is consistent
✓ All valid transaction records are preserved
Total missing values: 6

===== PROCESS COMPLETED =====

```

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

ANSWER:

To calculate **mean, median, and variance** efficiently and accurately, NumPy and Pandas aggregation functions can be used. Irrelevant or invalid values should be filtered before performing calculations.

1. Load and inspect the dataset

```
import pandas as pd
import numpy as np

df = pd.read_csv("data.csv")

print(df.head())
print(df.info())
```

2. Filter irrelevant data

Boolean filtering and indexing can remove invalid values before statistical calculations.

```
df = df[df['Value'].notna()]
df = df[df['Value'] >= 0]
```

`notna()` removes missing values, while the condition removes invalid negative values.

3. Calculate statistical measures

NumPy can calculate the required statistics efficiently:

```
data = df['Value'].to_numpy(dtype=np.float64)

mean = np.mean(data)
median = np.median(data)
variance = np.var(data)

print("Mean:", mean)
print("Median:", median)
print("Variance:", variance)
```


Using float64 helps reduce **precision loss**, especially when the dataset contains very large or very small numerical values.

4. Use Pandas aggregation

The same calculations can be performed together using Pandas:

```
result = df['Value'].agg(['mean', 'median', 'var'])

print(result)
```

This avoids writing separate Pandas operations for each statistic.

5. Avoid redundant computations

The data should be filtered **once** before calculating all statistics instead of repeatedly filtering it.

```
valid_data = df.loc[
    df['Value'].notna() & (df['Value'] >= 0),
    'Value'
]

print(valid_data.mean())
print(valid_data.median())
print(valid_data.var())
```

6. Debug incorrect results

If the statistical output is incorrect, check:

```
print("Number of records:", len(valid_data))
print("Missing values:", valid_data.isna().sum())
print("Minimum:", valid_data.min())
print("Maximum:", valid_data.max())
print("Data type:", valid_data.dtype)
```

Also verify the difference between **population variance** and **sample variance**:

```
np.var(data)      # Population variance
np.var(data, ddof=1) # Sample variance
```

Pandas var() uses sample variance (ddof=1) by default, so NumPy's default result may differ.

Conclusion

The efficient approach is to filter the data once, use appropriate indexing, convert to float64 for numerical accuracy, and use NumPy/Pandas aggregation functions. Checking data types, missing values, minimum/maximum values, record counts, and the ddof parameter helps identify incorrect statistical outputs.

CODE:

```
import pandas as pd

import numpy as np

data = {

    'Student_ID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],

    'Marks': [85, 90, 78, 95, np.nan, 88, -10, 92, 1000, 80]

}

df = pd.DataFrame(data)

print("===== ORIGINAL DATA =====")

... if valid_data.min() >= 0 and valid_data.max() <= 100:

    print("✓ All values are within valid range")

    if valid_data.dtype == 'float64':

        print("✓ Float64 used for better numerical precision")

        print("\n===== ANALYSIS COMPLETED =====")
```

OUTPUT:

```
print("\n===== ANALYSIS COMPLETED =====")

... ===== ORIGINAL DATA =====
      Student_ID  Marks
0           101    85.0
1           102    90.0
2           103    78.0
3           104    95.0
4           105     NaN
5           106    88.0
6           107   -10.0
7           108    92.0
8           109  1000.0
9           110    80.0

===== DATA INFORMATION =====
Number of records: 10
Data type: float64
Missing values: 1

===== VALID DATA =====
0     85.0
1     90.0
2     78.0
3     95.0
5     88.0
7     92.0
9     80.0
Name: Marks, dtype: float64

===== NUMPY DATA =====
[85. 90. 78. 95. 88. 92. 80.]

Mean using NumPy: 86.85714285714286
Median using NumPy: 88.0
Population Variance: 33.265306122448976
Sample Variance: 38.8095238095238

===== PANDAS STATISTICS =====
```

```

7      92.0
9      80.0
...    Name: Marks, dtype: float64

===== NUMPY DATA =====
[85. 90. 78. 95. 88. 92. 80.]

Mean using NumPy: 86.85714285714286
Median using NumPy: 88.0
Population Variance: 33.265306122448976
Sample Variance: 38.8095238095238

===== PANDAS STATISTICS =====
Mean using Pandas: 86.85714285714286
Median using Pandas: 88.0
Sample Variance using Pandas: 38.8095238095238

===== PANDAS AGGREGATION =====
count      7.000000
mean       86.857143
median     88.000000
var        38.809524
min        78.000000
max        95.000000
Name: Marks, dtype: float64

===== DEBUGGING CHECKS =====
Number of valid records: 7
Minimum value: 78.0
Maximum value: 95.0
Missing values after filtering: 0
Data type: float64

===== FINAL VALIDATION =====
✓ No missing values
✓ All values are within valid range
✓ Float64 used for better numerical precision

===== ANALYSIS COMPLETED =====

```

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

ANSWER:

A complete Pandas-based data analysis pipeline can be organized into **loading, cleaning, filtering, grouping, sorting, aggregation, and visualization** stages.

1. Loading multiple files

Use glob to find multiple CSV files and Pandas to read them.

```
import pandas as pd
import glob
files = glob.glob("data/*.csv")
dataframes = []
for file in files:
    try:
        df = pd.read_csv(file)
        dataframes.append(df)
    except Exception as e:
        print("Error reading:", file, e)

data = pd.concat(dataframes, ignore_index=True)
```

This avoids manually loading every file and allows corrupted files to be identified using try-except.

2. Cleaning and standardizing data

```
data.columns = data.columns.str.strip()

data['Sales'] = pd.to_numeric(
    data['Sales'], errors='coerce'
)

data = data.dropna(subset=['Sales'])
```

This handles inconsistent column names and converts incorrect numerical values into missing values that can be handled.

3. User-defined filtering

Boolean indexing can filter records based on conditions.

```
minimum_sales = 500
```

```
filtered = data[
    data['Sales'] >= minimum_sales
]
```

The value can be changed by the user without changing the rest of the pipeline.

4. Grouping and aggregation

```
summary = filtered.groupby(
    'Category',
    as_index=False
)['Sales'].sum()
```

This calculates total sales for each category.

5. Sorting

```
summary = summary.sort_values(
    by='Sales',
    ascending=False
).reset_index(drop=True)
```

```
print(summary)
```

Sorting makes it easier to identify the highest-performing categories.

6. Visualization

A bar chart can show the aggregated results clearly.

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8, 5))
```

plt.bar(

```
summary['Category'],  
summary['Sales']  
)
```

```
plt.xlabel('Category')  
plt.ylabel('Total Sales')  
plt.title('Sales by Category')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

7. Debugging

For corrupted files, use try-except while reading files. For inconsistent formats, check:

```
print(data.info())  
print(data.columns)  
print(data.isnull().sum())  
print(data.dtypes)
```

Unexpected values can be detected using filtering, and duplicate records can be checked using:

```
print(data.duplicated().sum())
```

Optimization

For large datasets, unnecessary columns can be avoided using usecols, appropriate data types can be specified with dtype, and large files can be processed in chunks using chunksize.

Conclusion

The pipeline is:

Read multiple files → Validate and clean → Filter → Group → Aggregate → Sort → Visualize → Validate output.

Pandas provides efficient DataFrame operations for processing and aggregation, while Matplotlib provides meaningful visualizations. Error handling, data-type checking, missing-value detection, and duplicate checking help identify corrupted or inconsistent input data.

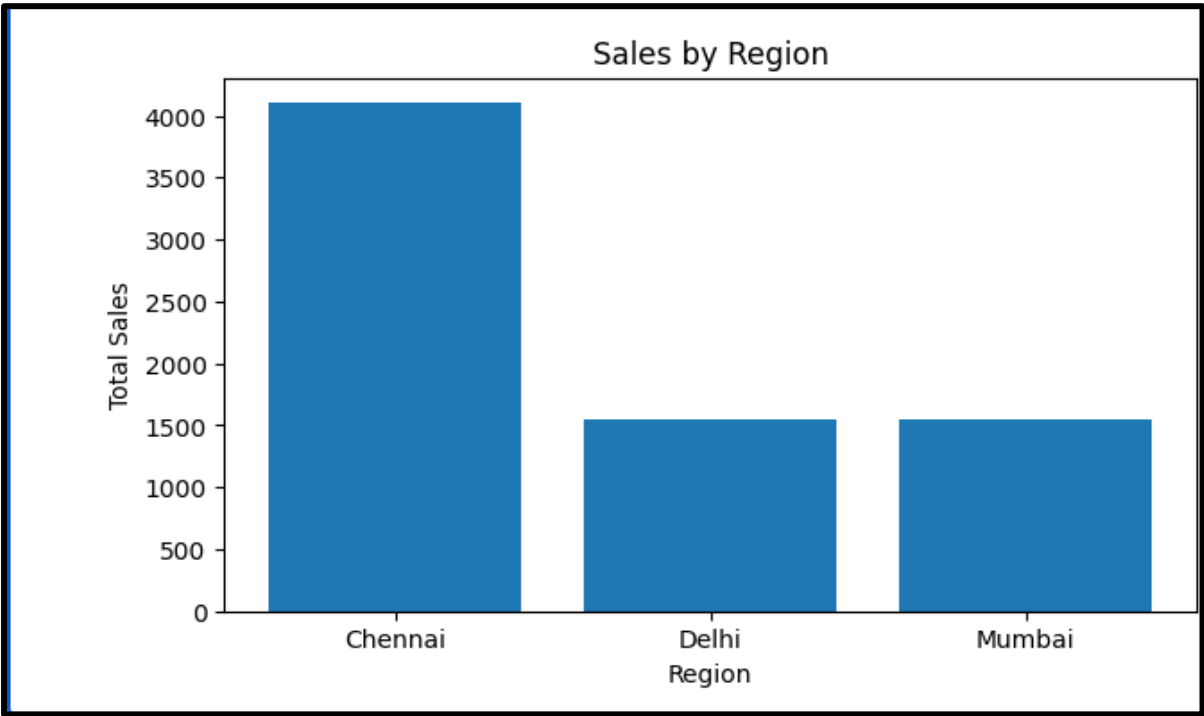
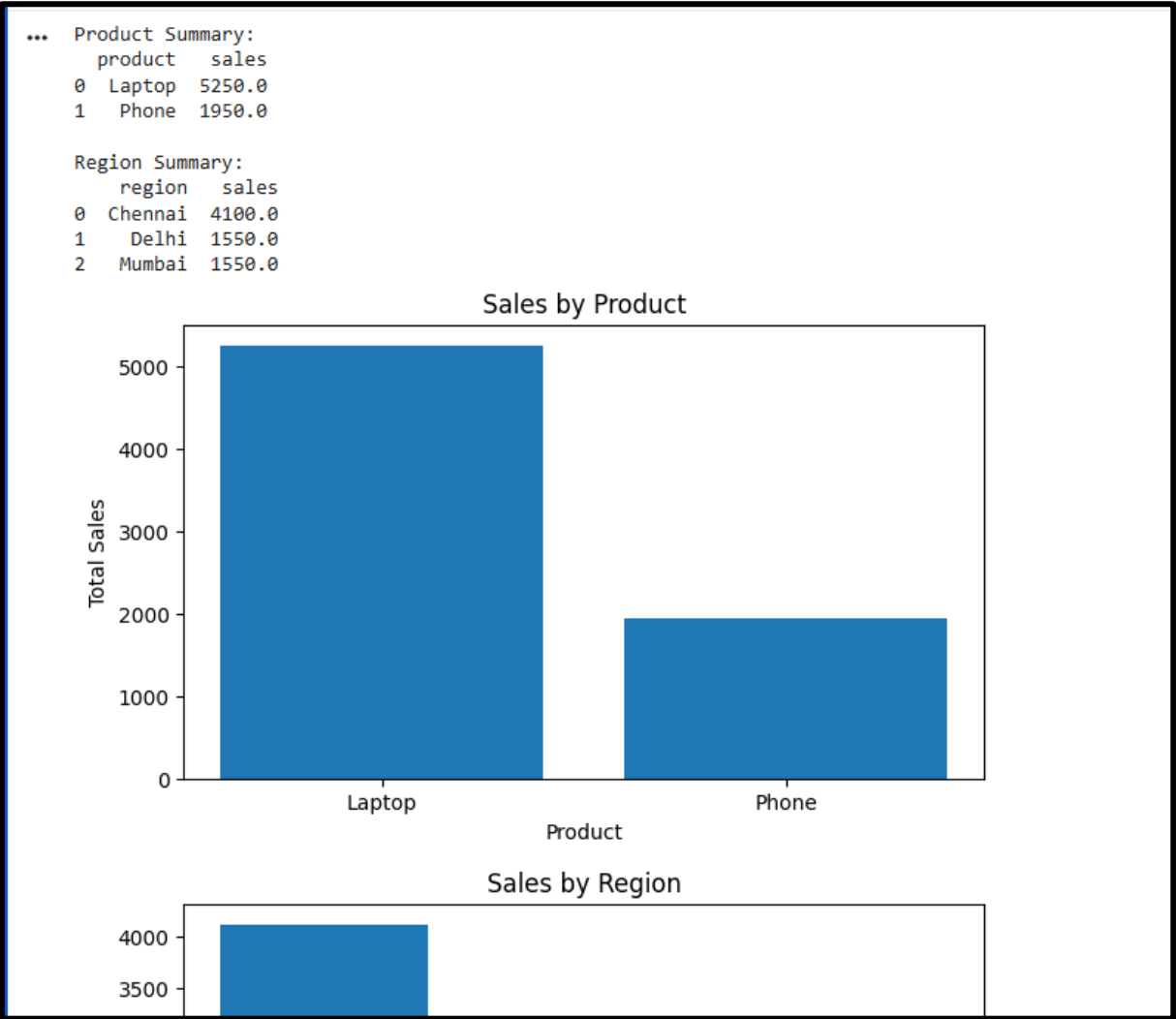
CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import glob
import os

# Create sample files
os.makedirs("data", exist_ok=True)

pd.DataFrame({
    'Product': ['Laptop', 'Phone', 'Tablet', 'Laptop', 'Phone'],
    ...plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.title("Sales by Region")
plt.show()
```


OUTPUT:



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

